

UNIVERSIDADE PAULISTA

LUCAS SOUZA OLIVEIRA (R845BJ3)
LUIZ HENRIQUE YAMAGUCHI SODRÉ (F3603A3)
NYEDSON PHELIPE MORAES MELO (R830349)
IGOR BATALHA BRANCO (F363DE2)
HENRY AUGUSTO SILVA DE SOUZA (H4186E3)

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
TURMA: DS4P17

APLICAÇÃO INTEGRADA A UM SISTEMA DE GERENCIAMENTO DE
BANCO DE DADOS RELACIONAL

SOROCABA

2025

1. INTRODUÇÃO

1.1 Objetivo

Este documento apresenta a documentação técnica do projeto CineManager, desenvolvido para a disciplina de Banco de Dados. O objetivo do projeto é desenvolver uma aplicação integrada a um Sistema de Gerenciamento de Banco de Dados Relacional (SGBDR), aplicando conceitos de modelagem de dados, integridade, operações CRUD e persistência.

O sistema foi desenvolvido para organizar informações relacionadas ao gerenciamento de um cinema, permitindo o controle de filmes, salas, sessões e ingressos. A aplicação também busca demonstrar, na prática, a integração entre a interface do sistema, a lógica de aplicação e o banco de dados. O sistema de cinema desenvolvido para a disciplina de Banco de Dados. O objetivo é projetar, documentar e implementar uma aplicação completa integrada a um SGBDR, contemplando modelagem relacional, integridade de dados, operações CRUD e persistência.

1.2 Escopo

O CineManager permite o gerenciamento de filmes, salas e sessões, além da realização de vendas e cancelamentos de ingressos. O sistema utiliza C# com ADO.NET, sem frameworks de back-end, e SQL Server como SGBD.

O front-end foi desenvolvido utilizando HTML5, CSS3, JavaScript, jQuery e Bootstrap, conforme as tecnologias permitidas na atividade. O escopo também contempla a implementação das operações CRUD, a persistência dos dados, o relacionamento entre as entidades do banco e as respectivas regras de integridade.

A documentação apresenta ainda a modelagem do banco de dados, o Diagrama Entidade-Relacionamento (DER), os scripts DDL, as principais funcionalidades do sistema e as instruções necessárias para instalação e execução do projeto.

2. DESCRIÇÃO DO PROJETO

2.1 Tema

O CineManager tem como tema a gestão de cinemas, com foco no gerenciamento de filmes, salas de exibição, sessões e ingressos. O sistema foi desenvolvido para centralizar essas informações e facilitar o controle da programação, permitindo relacionar filmes às respectivas salas e horários, além de realizar o gerenciamento da venda e do cancelamento de ingressos.

2.2 Regras de Negócio

O funcionamento do sistema é baseado nas seguintes regras de negócio:

- Cada filme possui título único, gênero, duração, classificação indicativa e sinopse.
- Cada sala possui nome único e capacidade máxima de 300 lugares.
- Cada sessão relaciona um filme a uma sala, em uma data e horário específicos, possuindo também preço e tipo de exibição (Dublado ou Legendado).
- Não é permitida a existência de duas sessões na mesma sala, na mesma data e horário.
- Cada ingresso está vinculado a uma sessão e registra os dados do cliente, número do assento, preço pago e status (Ativo ou Cancelado).
- Não pode existir mais de um ingresso Ativo para o mesmo assento em uma determinada sessão. Quando um ingresso é cancelado, o assento fica novamente disponível.
- O preço do ingresso é armazenado no momento da compra, garantindo a preservação do valor pago no histórico da venda.
- O CPF do cliente deve possuir exatamente 11 dígitos numéricos.

- O e-mail deve seguir um formato válido, contendo ao menos uma estrutura no padrão algo@algo.algo.
- A duração do filme deve ser maior que zero.
- A classificação indicativa deve corresponder a um dos valores permitidos: 0, 10, 12, 14, 16 ou 18 anos.

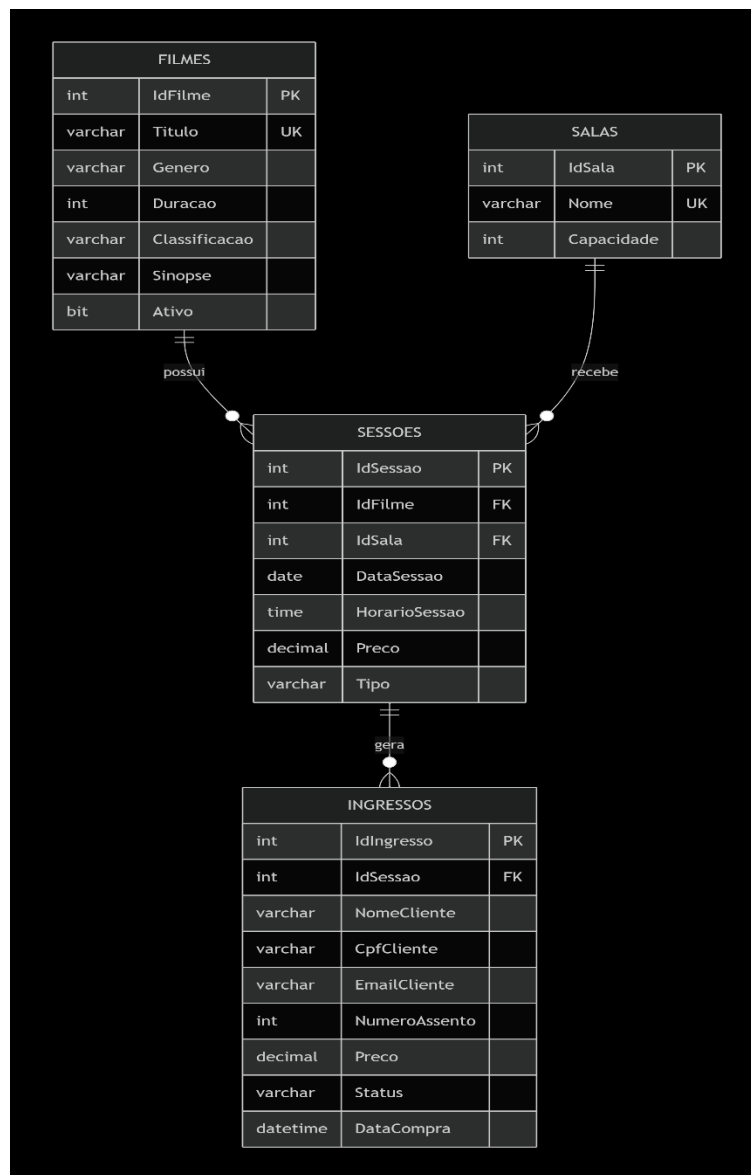
2.3 Funcionalidades

O CineManager disponibiliza as seguintes funcionalidades principais:

- CRUD de Filmes: cadastro, consulta, alteração e exclusão de filmes.
- CRUD de Salas: cadastro, consulta, alteração e exclusão de salas.
- CRUD de Sessões: cadastro, consulta, alteração e exclusão de sessões.
- Venda de ingressos: registro da compra de ingressos vinculados a uma sessão e a um assento.
- Cancelamento de ingressos: alteração do status do ingresso para cancelado, liberando o respectivo assento.
- Consulta de assentos disponíveis: visualização dos assentos que podem ser selecionados em uma determinada sessão.
- Listagem de sessões: consulta das sessões de acordo com filme, sala ou data.

3. MODELAGEM DE DADOS

3.1 Diagrama Entidade-Relacionamento (DER)



Descrição textual do DER:

- **Filmes (1) — (N) Sessões:** um filme pode ter várias sessões; cada sessão pertence a um único filme.
- **Salas (1) — (N) Sessões:** uma sala pode receber várias sessões; cada sessão ocorre em uma única sala.
- **Sessões (1) — (N) Ingressos:** uma sessão pode ter vários ingressos; cada ingresso pertence a uma única sessão.

3.2 Dicionário de Dados

Tabela: Filmes.

Campo	Tipo	Restrições	Descrição
IdFilme	INT	PK, IDENTITY	Identificador único do filme
Titulo	VARCHAR(150)	NOT NULL, UNIQUE, CHECK (len>0)	Título do filme
Genero	VARCHAR(50)	NOT NULL	Gênero cinematográfico
Duracao	INT	NOT NULL, CHECK (>0)	Duração em minutos
Classificacao	VARCHAR(5)	NOT NULL, CHECK (0,10,12,14,16,18)	Classificação indicativa
Sinopse	VARCHAR(500)	NULL	Resumo da trama
Ativo	BIT	NOT NULL, DEFAULT 1	Indica se o filme está em cartaz

Tabela: Sessoes.

Campo	Tipo	Restrições	Descrição
IdSessao	INT	PK, IDENTITY	Identificador único da sessão
IdFilme	INT	FK → Filmes, NOT NULL	Filme exibido
IdSala	INT	FK → Salas, NOT NULL	Sala de exibição
DataSessao	DATE	NOT NULL	Data da sessão
HorarioSessao	TIME(0)	NOT NULL	Horário da sessão
Preco	DECIMAL(10,2)	NOT NULL, CHECK (>=0)	Preço do ingresso
Tipo	VARCHAR(20)	NOT NULL, DEFAULT 'Dublado', CHECK (Dublado/Legendado)	Tipo de áudio
—	—	UNIQUE (IdSala, DataSessao, HorarioSessao)	Impede conflito de horário na mesma sala

Tabela: Ingressos.

Campo	Tipo	Restrições	Descrição
IdIngresso	INT	PK, IDENTITY	Identificador único do ingresso
IdSessao	INT	FK → Sessoes, NOT NULL	Sessão do ingresso
NomeCliente	VARCHAR(100)	NOT NULL, CHECK (len>=3)	Nome do cliente
CpfCliente	VARCHAR(11)	NOT NULL, CHECK (11 dígitos numéricos)	CPF do cliente
EmailCliente	VARCHAR(150)	NOT NULL, CHECK (formato e-mail)	E-mail do cliente
NumeroAssento	INT	NOT NULL, CHECK (>0)	Número do assento
Preco	DECIMAL(10,2)	NOT NULL, CHECK (>=0)	Preço pago no momento da compra
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Ativo', CHECK (Ativo/Cancelado)	Situação do ingresso
DataCompra	DATETIME	NOT NULL, DEFAULT GETDATE()	Data e hora da compra
—	—	UNIQUE INDEX filtrado (IdSessao, NumeroAssento) WHERE Status='Ativo'	Impede dois ingressos ativos para o mesmo assento na mesma sessão

3.3 Normalização

O modelo está na **Terceira Forma Normal (3FN)**, pois:

- Todos os atributos são atômicos (1FN).
- Não há dependências parciais, pois as chaves primárias são simples (2FN).
- Não há dependências transitivas: os dados de filme e sala não se repetem na tabela Sessoes; o preço do ingresso é histórico e não depende de outro atributo não-chave.

3.4 Script DDL

O script completo de criação do banco de dados, tabelas, restrições e dados iniciais encontra-se no **Anexo A** e no arquivo `BancoDados.sql` do repositório.

4. GUIA DE INSTALAÇÃO E EXECUÇÃO

4.1 Pré-requisitos

- Microsoft SQL Server (Express ou superior)
- SQL Server Management Studio (SSMS) ou Azure Data Studio
- .NET 8.0 SDK (para o back-end em C#)
- Navegador web moderno (para o front-end)
- Git (para clonar o repositório)

4.2 Configuração do Banco de Dados

1. Abra o SSMS e conecte-se à instância do SQL Server.
2. Execute o script `BancoDados.sql` (Anexo A) por completo.
3. Verifique se o banco `CineManager` foi criado e se as tabelas contêm os dados iniciais.

4.3 Configuração da Aplicação

1. Clone o repositório:

```
git clone https://github.com/[usuario]/CineManager.git
```
2. Abra a solução no Visual Studio.
3. Abra o arquivo `ConnectionString.txt` em qualquer editor de texto.
4. Na primeira linha do arquivo, informe a connection string correspondente à sua instância do SQL Server. Linhas iniciadas com `#` são ignoradas pelo programa e servem apenas como documentação/exemplos.

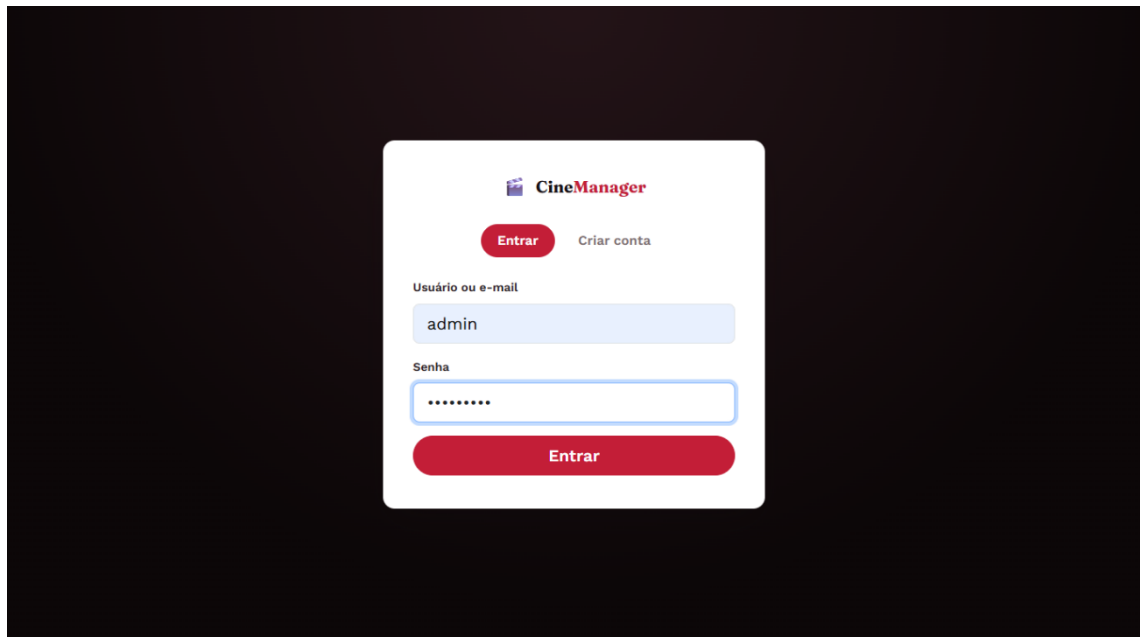
4.4 Execução

1. Abra o arquivo `CineManager.sln` no **Microsoft Visual Studio** (basta dar duplo clique no arquivo ou usar `Arquivo → Abrir → Projeto/Solução`).
2. Aguarde o carregamento da solução e a restauração das dependências.
3. No menu superior, selecione **Build** → **Build Solution** (`Ctrl + Shift + B`) para compilar o projeto.
4. Pressione `F5` (ou clique em **Start/Debug**) para executar a aplicação.
5. A interface do sistema será aberta e estará pronta para realizar as operações de CRUD e venda de ingressos.

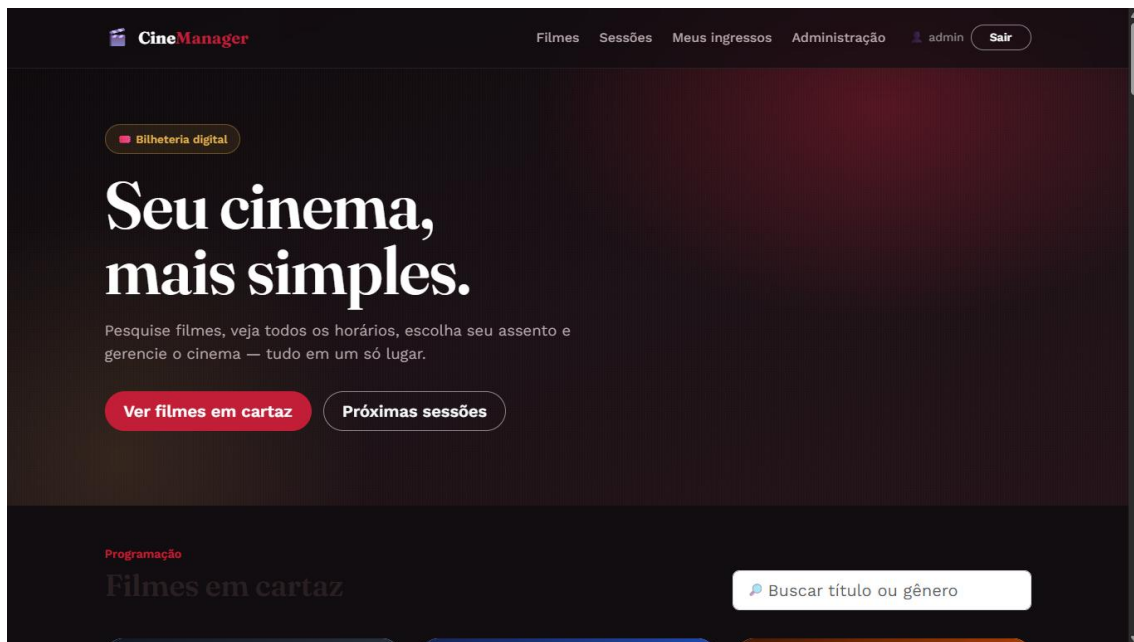
5. EVIDÊNCIAS VISUAIS

Capturas de tela do projeto.

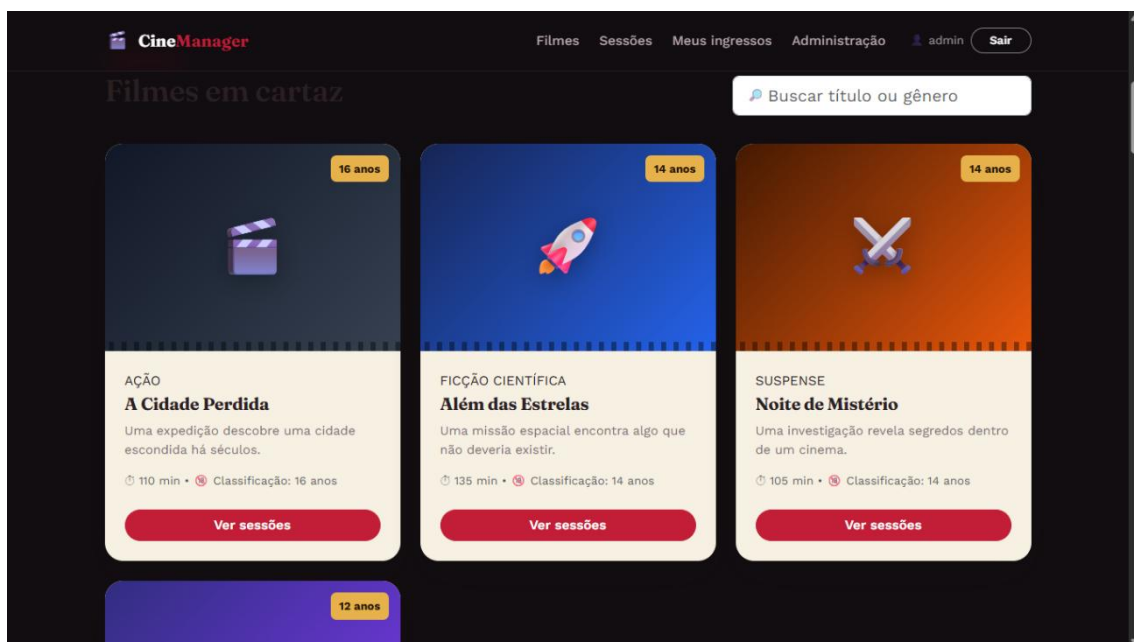
- Autenticação do sistema.



- Banner na tela inicial.



- Filmes em cartaz.



- Próximas sessões.

CineManager

FilmesSessõesMeus ingressosAdministraçãoadminSair

Horários

Próximas sessões

Todos os filmes

16
SET

A Cidade Perdida

Sala 3 • Dublado • R\$ 50,00

19:00 • Escolher

16
SET

O Último Guardião

Sala 1 • Dublado • R\$ 50,00

19:30 • Escolher

16
SET

Além das Estrelas

Sala 2 • Legendado • R\$ 50,00

20:00 • Escolher

16
SET

A Cidade Perdida

Sala 3 • Dublado • R\$ 50,00

21:30 • Escolher

16
SET

O Último Guardião

Sala 1 • Dublado • R\$ 50,00

21:45 • Escolher

16
SET

Além das Estrelas

Sala 2 • Legendado • R\$ 50,00

22:30 • Escolher

- Meus ingressos.

CineManager

FilmesSessõesMeus ingressosAdministraçãoadminSair

Horários

Próximas sessões

Todos os filmes

16
SET

A Cidade Perdida

Sala 3 • Dublado • R\$ 50,00

19:00 • Escolher

16
SET

O Último Guardião

Sala 1 • Dublado • R\$ 50,00

19:30 • Escolher

16
SET

Além das Estrelas

Sala 2 • Legendado • R\$ 50,00

20:00 • Escolher

16
SET

A Cidade Perdida

Sala 3 • Dublado • R\$ 50,00

21:30 • Escolher

16
SET

O Último Guardião

Sala 1 • Dublado • R\$ 50,00

21:45 • Escolher

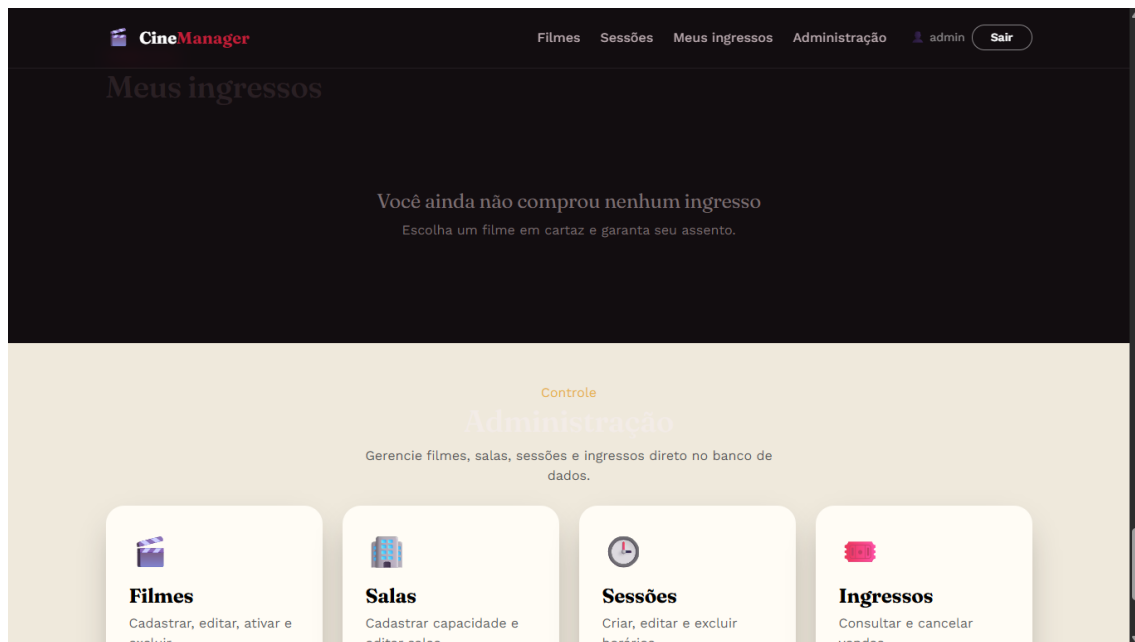
16
SET

Além das Estrelas

Sala 2 • Legendado • R\$ 50,00

22:30 • Escolher

- Administração.



ANEXO A – SCRIPT DDL COMPLETO

```
/* =====

CineManager - Script de criacao do banco de dados

SGBD: Microsoft SQL Server

Execute este arquivo inteiro no SQL Server Management Studio
(SSMS).

O script faz, nesta ordem:

1. cria o banco CineManager (se ainda nao existir);
2. seleciona o banco;
3. apaga as tabelas antigas (permite reexecutar o script);
4. cria as tabelas FILMES, SALAS, SESSOES e INGRESSOS;
5. cria PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK e DEFAULT;
6. insere os dados iniciais.

===== */

IF DB_ID('CineManager') IS NULL
BEGIN
    CREATE DATABASE CineManager;
END
GO

USE CineManager;
GO

/* -----

1) LIMPEZA

As tabelas sao apagadas da "filha" para a "mae" por causa
das chaves estrangeiras (Ingressos depende de Sessoes,
Sessoes depende de Filmes e Salas).

----- */

IF OBJECT_ID('dbo.Ingressos', 'U') IS NOT NULL DROP TABLE
dbo.Ingressos;

IF OBJECT_ID('dbo.Sessoes', 'U') IS NOT NULL DROP TABLE dbo.Sessoes;
IF OBJECT_ID('dbo.Salas', 'U') IS NOT NULL DROP TABLE dbo.Salas;
```

```

IF OBJECT_ID('dbo.Filmes', 'U') IS NOT NULL DROP TABLE dbo.Filmes;
IF OBJECT_ID('dbo.Usuarios', 'U') IS NOT NULL DROP TABLE
dbo.Usuarios;

GO

```

```

/* -----
2) TABELA USUARIOS

Entidade independente: guarda os usuarios do sistema de
login/cadastro. E-mail, CPF e NomeUsuario sao UNIQUE: o
proprio banco impede duplicidade, mesmo que alguém tente
inserir direto pelo SQL (o C# também valida antes de inserir,
mas a garantia final é sempre do banco).

A senha nunca é guardada em texto puro: o C# grava apenas o
hash (SHA-256) e o salt usados para gera-lo.

----- */
CREATE TABLE dbo.Usuarios
(
    IdUsuario      INT          IDENTITY(1,1) NOT NULL,
    NomeCompleto   VARCHAR(150) NOT NULL,
    NomeUsuario    VARCHAR(50)  NOT NULL,
    Email          VARCHAR(150) NOT NULL,
    Cpf            VARCHAR(11)  NOT NULL,
    SenhaHash      VARCHAR(200) NOT NULL,
    SenhaSalt      VARCHAR(200) NOT NULL,

    /* Identifica o papel do usuario no sistema. So quem tiver
       TipoUsuario = 'Administrador' pode acessar o painel
       administrativo (o C# também confere isso a cada requisicao,
       não é uma regra só do front-end). */
    TipoUsuario    VARCHAR(20)  NOT NULL CONSTRAINT
DF_Usuarios_TipoUsuario DEFAULT ('Comum'),

    /* Permite desativar um usuario sem apagar seu historico
       (ingressos
       comprados continuam existindo, ligados a este IdUsuario). Um
       usuario com Ativo = 0 não consegue mais fazer login - o C#
       confere isso em UsuarioRepositorio.Autenticar. */
    Ativo          BIT          NOT NULL CONSTRAINT DF_Usuarios_Ativo
DEFAULT (1),

```

```

        DataCadastro DATETIME          NOT NULL CONSTRAINT
DF_Usuarios_DataCadastro DEFAULT (GETDATE()),

        CONSTRAINT PK_Usuarios          PRIMARY KEY (IdUsuario),
        CONSTRAINT UQ_Usuarios_NomeUsuario UNIQUE (NomeUsuario),
        CONSTRAINT UQ_Usuarios_Email    UNIQUE (Email),
        CONSTRAINT UQ_Usuarios_Cpf      UNIQUE (Cpf),
        CONSTRAINT CK_Usuarios_NomeCompleto CHECK
(LEN(LTRIM(RTRIM(NomeCompleto))) > 0),
        CONSTRAINT CK_Usuarios_NomeUsuario CHECK
(LEN(LTRIM(RTRIM(NomeUsuario))) > 0),

        /* CPF precisa ter exatamente 11 caracteres e todos numericos. */
        CONSTRAINT CK_Usuarios_Cpf      CHECK (LEN(Cpf) = 11 AND Cpf
NOT LIKE '%[^0-9]%',

        /* Formato minimo de e-mail: alguma coisa @ alguma coisa . alguma
coisa */
        CONSTRAINT CK_Usuarios_Email    CHECK (Email LIKE
'%_@_%.__%'),
        CONSTRAINT CK_Usuarios_TipoUsuario CHECK (TipoUsuario IN
('Comum', 'Administrador'))
);
GO

/* -----
3) TABELA FILMES

Entidade independente: nao depende de nenhuma outra tabela.
----- */

CREATE TABLE dbo.Filmes
(
        IdFilme          INT          IDENTITY(1,1) NOT NULL,
        Titulo           VARCHAR(150) NOT NULL,
        Genero           VARCHAR(50)  NOT NULL,
        Duracao          INT          NOT NULL,
        Classificacao    VARCHAR(10)  NOT NULL,
        Sinopse          VARCHAR(500) NULL,
        Ativo            BIT          NOT NULL CONSTRAINT DF_Filmes_Ativo
DEFAULT (1),

        CONSTRAINT PK_Filmes          PRIMARY KEY (IdFilme),

```

```

        CONSTRAINT UQ_Filmes_Titulo          UNIQUE (Titulo),
        CONSTRAINT CK_Filmes_Duracao         CHECK (Duracao > 0),
        CONSTRAINT CK_Filmes_Titulo         CHECK
(LEN(LTRIM(RTRIM(Titulo))) > 0),

        /* Classificacao indicativa: um padrao unico de texto em todo o
           sistema (banco, backend e front-end), para nunca existir
           "12", "12 anos" e "12+" representando a mesma coisa. */
        CONSTRAINT CK_Filmes_Classificacao  CHECK (Classificacao IN
('Livre', '10 anos', '12 anos', '14 anos', '16 anos', '18
anos'))
);
GO

```

```

/* -----
4) TABELA SALAS

Tambem e uma entidade independente.
----- */

```

```

CREATE TABLE dbo.Salas
(
    IdSala      INT          IDENTITY(1,1) NOT NULL,
    Nome        VARCHAR(100) NOT NULL,
    Capacidade  INT          NOT NULL,

    CONSTRAINT PK_Salas          PRIMARY KEY (IdSala),
    CONSTRAINT UQ_Salas_Nome     UNIQUE (Nome),
    CONSTRAINT CK_Salas_Capacidade CHECK (Capacidade > 0 AND
Capacidade <= 300)
);
GO

```

```

/* -----
5) TABELA SESSOES

Tabela associativa: liga um FILME a uma SALA em uma data/hora.
Guarda apenas as chaves estrangeiras IdFilme e IdSala -
o titulo do filme e o nome da sala NAO sao repetidos aqui.
----- */

CREATE TABLE dbo.Sessoes

```

```

(
    IdSessao      INT          IDENTITY(1,1) NOT NULL,
    IdFilme       INT          NOT NULL,
    IdSala        INT          NOT NULL,
    DataSessao    DATE          NOT NULL,
    HorarioSessao TIME(0)      NOT NULL,
    Preco         DECIMAL(10,2) NOT NULL,
    Tipo          VARCHAR(20)   NOT NULL CONSTRAINT DF_Sessoes_Tipo
DEFAULT ('Dublado'),

    CONSTRAINT PK_Sessoes          PRIMARY KEY (IdSessao),
    CONSTRAINT FK_Sessoes_Filmes   FOREIGN KEY (IdFilme) REFERENCES
dbo.Filmes (IdFilme),
    CONSTRAINT FK_Sessoes_Salas    FOREIGN KEY (IdSala) REFERENCES
dbo.Salas (IdSala),

    /* O preco do ingresso inteiro e fixo em R$ 50,00 em todo o
        sistema (a meia-entrada, R$ 25,00, e calculada a partir deste
        valor no momento da compra - ver Backend/Precos.cs). Nenhuma
        sessao pode gravar um preco diferente, nem os valores antigos
        (R$ 30, R$ 32, R$ 35) usados antes desta regra existir. */
    CONSTRAINT CK_Sessoes_Precos   CHECK (Preco = 50.00),
    CONSTRAINT CK_Sessoes_Tipo     CHECK (Tipo IN ('Dublado',
'Legendado'))),

    /* Uma mesma sala nao pode ter duas sessoes no mesmo dia e
    horario. */
    CONSTRAINT UQ_Sessoes_SalaDataHorario UNIQUE (IdSala, DataSessao,
HorarioSessao)
);
GO

/* -----
5b) TRIGGER - horarios sem sobreposicao na mesma sala
Regra de negocio 10 do trabalho: a proxima sessao de uma sala
so pode comecar quando a sessao anterior daquela sala JA TIVER
TERMINADO (Inicio + Duracao do filme).

Isso e OBRIGATORIO tambem no banco (nao so no C#), porque um
INSERT/UPDATE feito direto no SQL (fora do backend) tem que
respeitar a mesma regra. O calculo usa SEMPRE a duracao REAL

```

do filme, vinda da tabela Filmes - nunca um horario fixo.

Conceito (em minutos desde 00:00, para nao depender do tipo
TIME do SQL Server em contas com horario):

```
        INICIO_NOVA < FIM_EXISTENTE  E  FIM_NOVA > INICIO_EXISTENTE
----- */
DROP TRIGGER IF EXISTS dbo.trg_Sessoes_ValidarConflitoHorario;
GO

CREATE TRIGGER dbo.trg_Sessoes_ValidarConflitoHorario
ON dbo.Sessoes
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM inserted AS Nova
        INNER JOIN dbo.Filmes AS FilmeDaNova
            ON FilmeDaNova.IdFilme = Nova.IdFilme
        INNER JOIN dbo.Sessoes AS Existente
            ON Existente.IdSala      = Nova.IdSala
            AND Existente.DataSessao = Nova.DataSessao
            AND Existente.IdSessao  <> Nova.IdSessao
        INNER JOIN dbo.Filmes AS FilmeDaExistente
            ON FilmeDaExistente.IdFilme = Existente.IdFilme
        WHERE
            -- inicio da nova, em minutos desde 00:00 < fim da
            existente
            (DATEPART(HOUR, Nova.HorarioSessao) * 60 +
             DATEPART(MINUTE, Nova.HorarioSessao))
            <
            (DATEPART(HOUR, Existente.HorarioSessao) * 60 +
             DATEPART(MINUTE, Existente.HorarioSessao)
              + FilmeDaExistente.Duracao)
        AND
```

```

        -- fim da nova, em minutos desde 00:00 > inicio da
existente

        (DATEPART(HOUR, Nova.HorarioSessao) * 60 +
DATEPART(MINUTE, Nova.HorarioSessao)

        + FilmeDaNova.Duracao)

        >

        (DATEPART(HOUR, Existente.HorarioSessao) * 60 +
DATEPART(MINUTE, Existente.HorarioSessao))

    )

BEGIN

    RAISERROR(

        'Não é possível cadastrar esta sessão porque o horário
entra em conflito com outra sessão da mesma sala.',

        16, 1);

    ROLLBACK TRANSACTION;

    RETURN;

END

END

GO

/* -----
6) TABELA INGRESSOS

Depende de SESSOES. Guarda o preco pago porque ele e um dado
historico: se o preco da sessao mudar depois, o ingresso ja
vendido deve continuar registrando o valor que o cliente pagou.
----- */

CREATE TABLE dbo.Ingressos

(

    IdIngresso      INT              IDENTITY(1,1) NOT NULL,

    IdSessao        INT              NOT NULL,

    /* Dono do ingresso: o usuario autenticado que realizou a compra.
    E por esta coluna - e NUNCA pelo CPF digitado no formulario -
    que o backend descobre quais ingressos pertencem a cada
    usuario (rota "Meus ingressos"). Fica NULL apenas nos
    registros de demonstracao inseridos por este script antes de
    existir usuario dono (ver secao 8); toda compra feita pelo
    sistema a partir de agora grava o IdUsuario da sessao logada.

    */

```

```

    IdUsuario      INT              NULL,
    NomeCliente    VARCHAR(100)    NOT NULL,
    CpfCliente     VARCHAR(11)     NOT NULL,
    EmailCliente   VARCHAR(150)    NOT NULL,
    NumeroAssento  INT              NOT NULL,
    Preco          DECIMAL(10,2)   NOT NULL,
    /* Tipo do ingresso vendido. Define o preco cobrado (o C# e quem
       decide o valor a partir daqui - ver Backend/Precos.cs -, o
       front-end nunca envia o preco). */
    TipoIngresso   VARCHAR(10)     NOT NULL CONSTRAINT
DF_Ingressos_TipoIngresso DEFAULT ('Inteira'),
    /* Documento que comprova o direito a meia-entrada (carteirinha
       estudantil, documento de idoso, etc.). So e preenchido quando
       TipoIngresso = 'Meia'; fica NULL na inteira. */
    TipoDocumentoMeia VARCHAR(50)  NULL,
    Status         VARCHAR(20)     NOT NULL CONSTRAINT
DF_Ingressos_Status DEFAULT ('Ativo'),
    DataCompra     DATETIME        NOT NULL CONSTRAINT
DF_Ingressos_DataCompra DEFAULT (GETDATE()),

    CONSTRAINT PK_Ingressos          PRIMARY KEY (IdIngresso),
    CONSTRAINT FK_Ingressos_Sessoes  FOREIGN KEY (IdSessao)
REFERENCES dbo.Sessoes (IdSessao),

    /* Liga o ingresso ao usuario autenticado que comprou. NULL e
       permitido apenas para nao quebrar registros historicos sem
       dono; o C# sempre preenche esta coluna a partir da sessao
       de login (GerenciadorDeSessoes), nunca a partir do que o
       navegador envia no corpo da requisicao. */
    CONSTRAINT FK_Ingressos_Usuarios FOREIGN KEY (IdUsuario)
REFERENCES dbo.Usuarios (IdUsuario),

    CONSTRAINT CK_Ingressos_Status  CHECK (Status IN ('Ativo',
'Cancelado')),
    CONSTRAINT CK_Ingressos_Assento  CHECK (NumeroAssento > 0),
    CONSTRAINT CK_Ingressos_Precos   CHECK (Preco >= 0),
    CONSTRAINT CK_Ingressos_Nome     CHECK
(LEN(LTRIM(RTRIM(NomeCliente))) >= 3),

    /* CPF precisa ter exatamente 11 caracteres e todos numericos. */
    CONSTRAINT CK_Ingressos_Cpf      CHECK (LEN(CpfCliente) = 11 AND
CpfCliente NOT LIKE '%[^0-9]%'),

```

```

/* Formato minimo de e-mail: alguma coisa @ alguma coisa . alguma
coisa */

CONSTRAINT CK_Ingressos_Email CHECK (EmailCliente LIKE
'%_@_%._%'),

CONSTRAINT CK_Ingressos_TipoIngresso CHECK (TipoIngresso IN
('Inteira', 'Meia')),

/* Regra de negocio principal do preco: inteira custa SEMPRE
R$ 50,00 e meia-entrada custa SEMPRE R$ 25,00. O banco garante
essa regra mesmo que, por algum bug, o C# tente gravar outro
valor - e a ultima linha de defesa contra preco manipulado. */
CONSTRAINT CK_Ingressos_PrecoporTipo CHECK (
(TipoIngresso = 'Inteira' AND Preco = 50.00) OR
(TipoIngresso = 'Meia' AND Preco = 25.00)
),

/* A meia-entrada exige documento probatorio; a inteira nao usa
(e nao pode ter) documento algum. */
CONSTRAINT CK_Ingressos_TipoDocumentoMeiaPorTipo CHECK (
(TipoIngresso = 'Meia' AND TipoDocumentoMeia IS NOT NULL
AND LEN(LTRIM(RTRIM(TipoDocumentoMeia))) > 0) OR
(TipoIngresso = 'Inteira' AND TipoDocumentoMeia IS NULL)
),

CONSTRAINT CK_Ingressos_TipoDocumentoMeiaValido CHECK (
TipoDocumentoMeia IS NULL OR
TipoDocumentoMeia IN ('Carteirinha estudantil', 'Documento de
identificação de idoso')
)
);

GO

/* -----
7) REGRA PRINCIPAL DO SISTEMA

Nao pode existir mais de um ingresso ATIVO para o mesmo
assento na mesma sessao.

Usamos um indice UNIQUE com filtro (WHERE Status = 'Ativo'),
assim ingressos CANCELADOS nao entram na regra e o assento
volta a ficar disponivel apos um cancelamento.

O proprio banco garante essa integridade, mesmo que duas

```

```

    pessoas tentem comprar o mesmo assento ao mesmo tempo.

    ----- */
CREATE UNIQUE INDEX UQ_Ingressos_AssentoAtivoPorSessao
    ON dbo.Ingressos (IdSessao, NumeroAssento)
    WHERE Status = 'Ativo';
GO

/* -----
8) DADOS INICIAIS
Sao os mesmos filmes, salas e sessoes que existiam no
JavaScript do front-end original.
----- */

INSERT INTO dbo.Filmes (Titulo, Genero, Duracao, Classificacao,
Sinopse, Ativo)
VALUES
    ('O Último Guardião', 'Aventura', 120, '12 anos', 'Um
aventureiro precisa proteger um antigo segredo.', 1),
    ('Além das Estrelas', 'Ficção Científica', 135, '14 anos', 'Uma
missão espacial encontra algo que não deveria existir.', 1),
    ('A Cidade Perdida', 'Ação', 110, '16 anos', 'Uma
expedição descobre uma cidade escondida há séculos.', 1),
    ('Noite de Mistério', 'Suspense', 105, '14 anos', 'Uma
investigação revela segredos dentro de um cinema.', 1);
GO

INSERT INTO dbo.Salas (Nome, Capacidade)
VALUES
    ('Sala 1', 40),
    ('Sala 2', 32),
    ('Sala 3', 48);
GO

/* As sessoes referenciam o filme e a sala por SUBCONSULTA,
e nao por numero fixo, para o script continuar correto
mesmo que os IDENTITY comecem em outro valor.

Sessoes espalhadas por VARIOS DIAS (16 a 21/09/2026), com os 4
filmes distribuidos entre as 3 salas e horarios diferentes a

```

cada dia. Nenhum horario se sobrepoe a outro na mesma sala: o inicio de cada sessao so acontece depois do FIM (inicio + duracao) da sessao anterior naquela sala - a mesma regra que o trigger trg_Sessoes_ValidarConflitoHorario (secao 5b) valida sozinho a cada INSERT.

Duracao de cada filme, para conferencia dos horarios abaixo:

O Último Guardião = 120 min A Cidade Perdida = 110 min
Além das Estrelas = 135 min Noite de Mistério = 105 min */

```
INSERT INTO dbo.Sessoes (IdFilme, IdSala, DataSessao, HorarioSessao,
Preco, Tipo)
```

```
VALUES
```

```
/* ----- 16/09/2026 ----- */
-- Sala 1: O Último Guardião (120min) 19:30-21:30, depois 21:45-
23:45

((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'O Último
Guardião'),

(SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 1'), '2026-
09-16', '19:30', 50.00, 'Dublado'),

((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'O Último
Guardião'),

(SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 1'), '2026-
09-16', '21:45', 50.00, 'Dublado'),

-- Sala 2: Além das Estrelas (135min) 20:00-22:15, depois 22:30-
00:45

((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Além das
Estrelas'),

(SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 2'), '2026-
09-16', '20:00', 50.00, 'Legendado'),

((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Além das
Estrelas'),

(SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 2'), '2026-
09-16', '22:30', 50.00, 'Legendado'),

-- Sala 3: A Cidade Perdida (110min) 19:00-20:50, depois 21:30-
23:20

((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'A Cidade
Perdida'),

(SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 3'), '2026-
09-16', '19:00', 50.00, 'Dublado'),
```

```

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'A Cidade
Perdida')),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 3'), '2026-
09-16', '21:30', 50.00, 'Dublado'),

/* ----- 17/09/2026 ----- */

-- Sala 1: Noite de Mistério (105min) 15:00-16:45, depois Além das
Estrelas (135min) 18:00-20:15

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Noite de
Mistério')),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 1'), '2026-
09-17', '15:00', 50.00, 'Dublado'),

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Além das
Estrelas')),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 1'), '2026-
09-17', '18:00', 50.00, 'Legendado'),

-- Sala 2: O Último Guardião (120min) 16:00-18:00, depois A Cidade
Perdida (110min) 19:00-20:50

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'O Último
Guardião')),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 2'), '2026-
09-17', '16:00', 50.00, 'Dublado'),

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'A Cidade
Perdida')),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 2'), '2026-
09-17', '19:00', 50.00, 'Legendado'),

-- Sala 3: Além das Estrelas (135min) 15:30-17:45, depois Noite de
Mistério (105min) 20:00-21:45

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Além das
Estrelas')),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 3'), '2026-
09-17', '15:30', 50.00, 'Legendado'),

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Noite de
Mistério')),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 3'), '2026-
09-17', '20:00', 50.00, 'Dublado'),

/* ----- 18/09/2026 ----- */

-- Sala 1: O Último Guardião (120min) 14:00-16:00, depois A Cidade
Perdida (110min) 18:00-19:50

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'O Último
Guardião')),

```

```

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 1'), '2026-
09-18', '14:00', 50.00, 'Dublado'),

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'A Cidade
Perdida'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 1'), '2026-
09-18', '18:00', 50.00, 'Dublado'),

-- Sala 2: Noite de Mistério (105min) 15:00-16:45, depois O Último
Guardião (120min) 19:00-21:00

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Noite de
Mistério'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 2'), '2026-
09-18', '15:00', 50.00, 'Legendado'),

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'O Último
Guardião'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 2'), '2026-
09-18', '19:00', 50.00, 'Legendado'),

-- Sala 3: A Cidade Perdida (110min) 16:00-17:50, depois Além das
Estrelas (135min) 20:00-22:15

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'A Cidade
Perdida'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 3'), '2026-
09-18', '16:00', 50.00, 'Dublado'),

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Além das
Estrelas'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 3'), '2026-
09-18', '20:00', 50.00, 'Dublado'),

/* ----- 19/09/2026 ----- */

-- Sala 1: Além das Estrelas (135min) 16:00-18:15, depois Noite de
Mistério (105min) 20:00-21:45

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Além das
Estrelas'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 1'), '2026-
09-19', '16:00', 50.00, 'Legendado'),

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Noite de
Mistério'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 1'), '2026-
09-19', '20:00', 50.00, 'Legendado'),

-- Sala 2: A Cidade Perdida (110min) 17:00-18:50, depois O Último
Guardião (120min) 20:00-22:00

```

```

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'A Cidade
Perdida'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 2'), '2026-
09-19', '17:00', 50.00, 'Dublado'),

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'O Último
Guardião'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 2'), '2026-
09-19', '20:00', 50.00, 'Dublado'),

-- Sala 3: O Último Guardião (120min) 15:00-17:00, depois A Cidade
Perdida (110min) 19:30-21:20

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'O Último
Guardião'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 3'), '2026-
09-19', '15:00', 50.00, 'Legendado'),

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'A Cidade
Perdida'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 3'), '2026-
09-19', '19:30', 50.00, 'Legendado'),

/* ----- 20/09/2026 ----- */

-- Sala 1: A Cidade Perdida (110min) 15:30-17:20, depois O Último
Guardião (120min) 19:00-21:00

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'A Cidade
Perdida'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 1'), '2026-
09-20', '15:30', 50.00, 'Dublado'),

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'O Último
Guardião'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 1'), '2026-
09-20', '19:00', 50.00, 'Dublado'),

-- Sala 2: Além das Estrelas (135min) 14:30-16:45, depois Noite de
Mistério (105min) 18:00-19:45

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Além das
Estrelas'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 2'), '2026-
09-20', '14:30', 50.00, 'Legendado'),

        ((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Noite de
Mistério'),

        (SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 2'), '2026-
09-20', '18:00', 50.00, 'Legendado'),

```

```

-- Sala 3: Noite de Mistério (105min) 16:00-17:45, depois Além das
Estrelas (135min) 19:30-21:45

((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Noite de
Mistério'),

(SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 3'), '2026-
09-20', '16:00', 50.00, 'Dublado'),

((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Além das
Estrelas'),

(SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 3'), '2026-
09-20', '19:30', 50.00, 'Dublado'),

/* ----- 21/09/2026 ----- */

-- Sala 1: O Último Guardião (120min) 14:30-16:30, depois Além das
Estrelas (135min) 19:00-21:15

((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'O Último
Guardião'),

(SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 1'), '2026-
09-21', '14:30', 50.00, 'Legendado'),

((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Além das
Estrelas'),

(SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 1'), '2026-
09-21', '19:00', 50.00, 'Legendado'),

-- Sala 2: A Cidade Perdida (110min) 15:00-16:50, depois Noite de
Mistério (105min) 18:30-20:15

((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'A Cidade
Perdida'),

(SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 2'), '2026-
09-21', '15:00', 50.00, 'Dublado'),

((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Noite de
Mistério'),

(SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 2'), '2026-
09-21', '18:30', 50.00, 'Dublado'),

-- Sala 3: Além das Estrelas (135min) 16:00-18:15, depois O Último
Guardião (120min) 20:30-22:30

((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'Além das
Estrelas'),

(SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 3'), '2026-
09-21', '16:00', 50.00, 'Legendado'),

((SELECT IdFilme FROM dbo.Filmes WHERE Titulo = 'O Último
Guardião'),

(SELECT IdSala FROM dbo.Salas WHERE Nome = 'Sala 3'), '2026-
09-21', '20:30', 50.00, 'Legendado');

```

GO

```
/* Usuario de demonstracao para testar o login (usuario COMUN,
sem acesso ao painel administrativo).
Usuario: demo | Senha: demo123
O CPF abaixo tem os digitos verificadores validos (algoritmo
oficial), a mesma regra que Validacoes.CpfValido passou a exigir.
O hash abaixo foi gerado pelo proprio C# (SHA-256 + salt); veja
Backend/Repositorios/UsuarioRepositorio.cs para o algoritmo. */
INSERT INTO dbo.Usuarios (NomeCompleto, NomeUsuario, Email, Cpf,
SenhaHash, SenhaSalt, TipoUsuario)
VALUES
('Usuário Demonstração', 'demo', 'demo@cinemanager.com',
'12345678909',
'NvgUBAwKXIrmFyhvLFqVr4ViFKH/PPV8BF5Zq6yWKno=',
'ZmFrZS1zYWx0LWRlbW8=', 'Comum');
GO
```

```
/* Usuario ADMINISTRADOR padrao, exigido pelo trabalho.
Usuario: admin | Senha: Admin@123
O CPF abaixo (111.444.777-35) tambem tem os digitos verificadores
validos - e um CPF classico usado como exemplo em material
didatico de validacao, nunca um CPF real de alguem.
(mesma logica de hash acima: SHA-256(salt + senha), gerado pelo
proprio algoritmo do C#). Documentado tambem no README.md. */
INSERT INTO dbo.Usuarios (NomeCompleto, NomeUsuario, Email, Cpf,
SenhaHash, SenhaSalt, TipoUsuario)
VALUES
('admin', 'admin', 'admin@gmail.com', '11144477735',
'62KQ1UPOTaoKIO6vdroz3IHUcge4KWFeMJKLAJUojcQ=',
's9ELVZJ7xJZ4pfC33GFJow==', 'Administrador');
GO
```

```
/* Ingressos de demonstracao: um inteira e um meia-entrada,
ja com os precos oficiais (R$ 50,00 / R$ 25,00) e com CPFs que
tambem passam pelos digitos verificadores do CPF. Ambos ficam
vinculados ao usuario "demo" (IdUsuario), o mesmo padrao que o
C# usa para toda compra feita por um usuario autenticado - assim
```

```

    dá para testar a tela "Meus ingressos" logando como demo. */

INSERT INTO dbo.Ingressos (IdSessao, IdUsuario, NomeCliente,
CpfCliente, EmailCliente, NumeroAssento, Preco, TipoIngresso,
TipoDocumentoMeia, Status)

VALUES

    ((SELECT MIN(IdSessao) FROM dbo.Sesoes),

    (SELECT IdUsuario FROM dbo.Usuarios WHERE NomeUsuario = 'demo'),

    'Cliente Demonstração', '19283746546', 'demo@email.com', 12,
50.00, 'Inteira', NULL, 'Ativo');

GO

INSERT INTO dbo.Ingressos (IdSessao, IdUsuario, NomeCliente,
CpfCliente, EmailCliente, NumeroAssento, Preco, TipoIngresso,
TipoDocumentoMeia, Status)

VALUES

    ((SELECT MIN(IdSessao) FROM dbo.Sesoes),

    (SELECT IdUsuario FROM dbo.Usuarios WHERE NomeUsuario = 'demo'),

    'Cliente Meia-Entrada', '98765432100', 'meia@email.com', 13,
25.00, 'Meia', 'Carteirinha estudantil', 'Ativo');

GO

/* -----
9) CONFERENCIA RAPIDA
----- */

SELECT 'Usuarios' AS Tabela, COUNT(*) AS TotalDeRegistros FROM
dbo.Usuarios

UNION ALL

SELECT 'Filmes', COUNT(*) FROM dbo.Filmes

UNION ALL

SELECT 'Salas', COUNT(*) FROM dbo.Salas

UNION ALL

SELECT 'Sesoes', COUNT(*) FROM dbo.Sesoes

UNION ALL

SELECT 'Ingressos', COUNT(*) FROM dbo.Ingressos;

GO

PRINT 'Banco CineManager criado com sucesso.';

GO

```